

LAB PROJECT REPORT

This report is present on the partial fulfillment of the course CSE:124 Data Structure Lab in the department of computer science and engineering.

Submitted By

Student Name	Student ID
Md Habibur Bashir srabon	252-15-741
Sree Rahul Kumar Gosh	252-15-667
Suresh Barua	252-15-711
Arpon Dam	251-15-269
Md Abid Hasan	251-15-132

LAB PROJECT REPORT

This Report presented in Partial Fulfillment of the course
CSE124: Data Structure Lab in the Department of Computer Science and
Engineering



DECLARATION

We hereby declare that this lab project titled "Smart Restaurant Management System" has been designed and developed by us under the supervision of our course Taslima Akhter, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere for any academic purpose.

Submitted To:

Taslima Akhter

Department of Computer Science and Engineering

Daffodil International University

Submitted by

Md Habibur Bashir srabon 252-15-741 DIU,CSE	
Sree Rahul Kumar Gosh 252-15-667 DIU,CSE	Suresh Barua 252-15-711 DIU,CSE
Arpon Dam 252-15-269 DIU,CSE	Md Abid Hasan 252-15-132 DIU,CSE

Course & Program OutcomeCourse Outcome Statements

CO's	Statements
CO1	Design the concept of fundamentals of data structure for solving real-life problem having complex engineering attributes.
CO2	Solve a real-life problem having application of abstract data type created within the scope of complex engineering problem solving with modern tools.
CO3	Apply Data Structure to solve real world problems as a member in diverse team.

Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO3	C1, C2	KP5	EP1
CO2	PO5	C3	KP6	EP2, EP6
CO3	PO9	C3	N/A	N/A

Table of Contents

- **Declaration – i**
- **Course & Program Outcome – ii**

Chapter 1: Introduction – 5

- 1.1 Introduction – 5
- 1.2 Motivation – 5
- 1.3 Objectives – 5
- 1.4 Feasibility Study – 5
- 1.5 Gap Analysis – 6
- 1.6 Project Outcome – 6

Chapter 2: System Architecture – 6

- 2.1 Requirement Analysis & Design Specification – 6
- 2.2 Use of Modern Tools – 9

Chapter 3: Implementation and Results – 9

- 3.1 Implementation – 9
- 3.2 Output – 12
- 3.3 Discussion – 12

Chapter 4: Engineering Standards and Mapping – 12

- 4.1 Impact on Society, Environment and Sustainability – 13
- 4.2 Complex Engineering Problem – 13
- 4.3 Mapping of Program Outcome – 14

Chapter 5: Conclusion – 16

- 5.1 Summary – 17
- 5.2 Limitation – 17
- 5.3 Future Work – 18

- **References – 18**

Chapter 1: Introduction

1.1 Introduction

The Smart Restaurant Management System (SRMS) is a console-based application developed in the C programming language as part of CSE124: Data Structure Lab at Daffodil International University. The system simulates the core operations of a modern restaurant using singly linked lists as the primary data structure.

The system supports two primary user roles: Customer and Authority/Admin. Customers can view the menu, place orders, request new food items, leave reviews, and view available special offers. The admin can manage menu items, update stock levels, view customer feedback, and handle food requests — all secured through password authentication.

1.2 Motivation

The primary motivation is to bridge the gap between theoretical knowledge of data structures and their practical application in real-world systems. Restaurants represent an ideal domain because they involve dynamic data — menus change, orders arrive, reviews accumulate — that must be efficiently stored and manipulated using linked lists and dynamic memory allocation.

1.3 Objectives

- Implement a fully functional restaurant management system using singly linked lists in C.
- Demonstrate dynamic memory allocation and pointer manipulation.
- Design a dual-role system (Customer and Authority) with appropriate access controls.
- Implement CRUD operations on menu items using linked list traversal.
- Simulate order processing, billing (including discount logic), and stock management.
- Collect and display customer reviews using append-only linked list operations.
- Apply password-based authentication for the admin module.
- Deliver a clear, intuitive console interface with unambiguous feedback.

1.4 Feasibility Study

Technical Feasibility

The system is implemented in C with no external libraries beyond the standard library (<stdio.h>, <stdlib.h>, <string.h>), making it fully portable and executable in any standard C development environment.

Operational Feasibility

The dual-role design (Customer/Admin) mirrors real restaurant staffing. The menu-driven interface requires minimal training and operates efficiently in a single-process, single-user console environment.

Schedule Feasibility

The project was completed within the allotted CSE124 laboratory course time. Tasks were divided among team members using a project plan, ensuring timely completion of all modules.

1.5 Gap Analysis

Existing Limitation	SRMS Solution
No dynamic menu modification	Admin can add/remove/update items at runtime
No stock tracking	Real-time quantity decrement on each order
No review mechanism	Customer review appended to linked list post-payment

No new-item feedback channel	Customer food-request list visible to admin
Complex GUI dependencies	Pure console — zero external dependencies, fully portable

1.6 Project Outcome

- A working console application with four independent linked lists.
- Admin authentication gate preventing unauthorized menu changes.
- Automatic 10% discount applied to orders exceeding 500 BDT.
- Clean memory deallocation for all heap-allocated nodes on exit.

Chapter 2: Proposed Methodology / Architecture

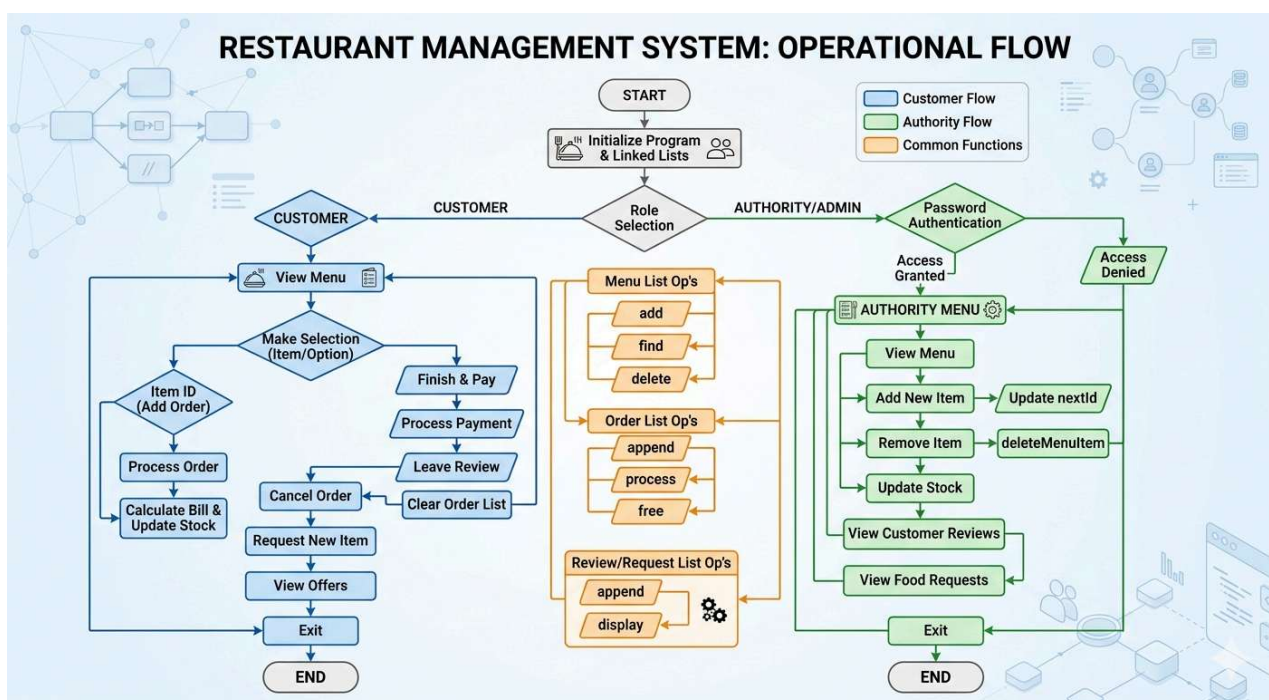
2.1 System Architecture Overview

The SRMS is architected around four singly linked lists, each managing a distinct domain of restaurant data. All lists persist in heap memory and share the same process lifetime. A role-selection gateway at startup determines which workflow — customer or admin — the process executes.

Linked List	Node Structure	Operations	Scope
MenuItem	id, name, price, quantity, *next	append, findById, delete, update	Global — persists session
Order	itemId, quantity, *next	append, processAll, freeAll	Per-customer transaction
Review	rating, feedback[200], *next	append, displayAll	Accumulates all sessions
Request	name[50], suggestedPrice, *next	append, displayAll	Accumulates all sessions

2.2 System Flowchart

The flowchart below depicts the complete execution flow from startup to exit, covering both the customer and authority branches, all menu choices, and the data structure operations triggered at each step.



2.3 UI Design — Console Screen Mockups

The following screens represent the actual console UI output of the system. Each screen corresponds to a specific state in the application workflow.

Screen 1: Role Selection (Entry Point)

```
[ Entry Screen – Role Selector ]
```

```

===== RESTAURANT SYSTEM =====
Select Role:
1. Customer
2. Authority (Admin)
Enter choice: _

```

Screen 2: Customer Welcome & Main Menu

```

[ Customer Flow - Menu Display ]

=====
WELCOME TO OUR RESTAURANT
=====
We are happy to serve you today!
=====

===== MENU =====
[1] Pizza           250.00 BDT (Stock: 10)
[2] Burger          150.00 BDT (Stock: 15)
[3] Pasta           200.00 BDT (Stock: 12)
[4] Sandwich        120.00 BDT (Stock: 20)
[5] Coffee           80.00 BDT (Stock: 30)
-----
[ 0] Finish & Pay
[-1] Cancel Order
[-2] Request a New Item
[-3] View Special Offers
=====
Enter choice: _

```

Screen 3: Adding Items to Order

```

[ Customer Flow - Ordering Items ]

Enter choice: 1
Enter quantity: 2
'Pizza' x2 added to your order.

Enter choice: 3
Enter quantity: 1
'Pasta' x1 added to your order.

Enter choice: 5
Enter quantity: 3
'Coffee' x3 added to your order.

```

Screen 4: Finishing & Paying — Bill Details

```

[ Customer Flow - Bill + Review ]

Enter choice: 0

===== BILL DETAILS =====
2 x Pizza           = 500.00 BDT
1 x Pasta            = 200.00 BDT
3 x Coffee           = 240.00 BDT
Discount (10% on orders > 500 BDT): -94.00 BDT
-----
TOTAL : 846.00 BDT
=====
Thank you for your order!

```

```
Rate our service (1-5): 5
Write your feedback: Excellent food, great service!
Review saved. See you again!
```

Screen 5: Special Offers

[Customer Flow – Special Offers]

Enter choice: -3

```
===== SPECIAL OFFERS =====
1. Buy 2 Pizzas, get 1 Free Drink!
2. 10% off on orders above 500 BDT.
3. Combo: Burger + Coffee = 200 BDT
=====
```

Screen 6: Requesting a New Food Item

[Customer Flow – Food Request]

```
Enter choice: -2
Enter food name to request: Chicken Shawarma
Suggest a price (BDT): 180
Request submitted! Thank you.
```

Screen 7: Authority Login

[Admin Flow – Login & Menu]

Select Role: 2

```
Enter authority password: admin123
Access granted.
```

```
===== AUTHORITY MENU =====
1. View Menu
2. Add New Menu Item
3. Remove Menu Item
4. Update Stock
5. View Customer Reviews
6. View Food Requests
0. Exit
=====
```

Enter option: _

Screen 8: Admin — Add New Menu Item

[Admin Flow – Add Item]

```
Enter option: 2
Enter item name: Chicken Shawarma
Enter price (BDT): 180
Enter initial stock: 25
Item 'Chicken Shawarma' added with ID 6.
```

Screen 9: Viewing Customer Reviews

[Admin Flow – Customer Reviews]

Enter option: 5

```

===== CUSTOMER REVIEWS =====
Review #1
Rating : ***** (5/5)
Feedback: Excellent food, great service!

Review #2
Rating : **** (4/5)
Feedback: Food was good but a bit slow.

=====

```

Screen 10: Viewing Food Requests

[Admin Flow – Food Requests]

Enter option: 6

```

===== REQUESTED FOOD ITEMS =====
1. Chicken Shawarma          Suggested: 180.00 BDT
2. Mango Lassi               Suggested: 90.00 BDT
=====

```

2.4 Overall Project Plan

Phase	Task	Members	Duration
1	Requirement Analysis & Design	All members	Day 1–2
2	Data Structure Design (Struct definitions)	Srabon, Barua	Day 3
3	Menu CRUD (Linked List ops)	Rahul, Abid	Day 4–5
4	Order & Billing Module	Srabon, Arpon	Day 6–7
5	Review & Request Module	Barua, Rahul	Day 8
6	Authentication & Admin Flow	Abid, Arpon	Day 9
7	Testing & Debugging	All members	Day 10
8	Documentation & Report	All members	Day 11–12

Chapter 3: Implementation and Results

3.1 Source Code

The complete annotated source code of the Smart Restaurant Management System is presented below. The system is organized into functional sections: structure definitions, linked list operations, flow controllers, and the main entry point.

3.1.1 Header Includes & Authentication Constant

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define AUTH_PASSWORD "admin123"
/* Password for admin access - compared using strcmp() */
```

3.1.2 Structure Definitions (Linked List Nodes)

Four distinct structures are defined, each acting as a node in its own singly linked list. The *next pointer enables chaining.

```
/* ---- MenuItem Node ---- */
typedef struct MenuItem {
    int    id;           /* unique item identifier */
    char   name[50];     /* food item name          */
    float  price;        /* price in BDT            */
    int    quantity;     /* current stock           */
    struct MenuItem *next; /* pointer to next node */
} MenuItem;

/* ---- Order Node (per-session) ---- */
typedef struct Order {
    int itemId;          /* references MenuItem.id */
    int quantity;        /* how many ordered       */
    struct Order *next;
} Order;

/* ---- Review Node ---- */
typedef struct Review {
    int    rating;       /* 1-5 stars              */
    char   feedback[200]; /* free-text comment      */
    struct Review *next;
} Review;

/* ---- Request Node ---- */
typedef struct Request {
    char   name[50];
    float  suggestedPrice;
    struct Request *next;
} Request;
```

3.1.3 Menu Linked List Operations

```
/* --- Create a new MenuItem node --- */
MenuItem *createMenuItem(int id, const char *name,
                        float price, int qty) {
    MenuItem *node = (MenuItem *)malloc(sizeof(MenuItem));
    node->id        = id;
    strncpy(node->name, name, 49);
    node->name[49] = '\0';
    node->price     = price;
    node->quantity  = qty;
    node->next      = NULL;
    return node;
}
```

```

}

/* --- Append to tail of menu list --- */
void appendMenuItem(MenuItem **head, int id,
                    const char *name, float price, int qty) {
    MenuItem *node = createMenuItem(id, name, price, qty);
    if (*head == NULL) { *head = node; return; }
    MenuItem *temp = *head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = node;
}

/* --- Find by ID (returns NULL if not found) --- */
MenuItem *findMenuItemById(MenuItem *head, int id) {
    while (head != NULL) {
        if (head->id == id) return head;
        head = head->next;
    }
    return NULL;
}

/* --- Delete by ID; returns 1 on success --- */
int deleteMenuItem(MenuItem **head, int id) {
    if (*head == NULL) return 0;
    if ((*head)->id == id) { /* head deletion */
        MenuItem *temp = *head;
        *head = (*head)->next;
        free(temp); return 1;
    }
    MenuItem *prev = *head, *curr = (*head)->next;
    while (curr != NULL) {
        if (curr->id == id) {
            prev->next = curr->next;
            free(curr); return 1;
        }
        prev = curr; curr = curr->next;
    }
    return 0;
}

```

3.1.4 Order Processing with Discount Logic

```

float processOrders(Order *head, MenuItem *menuHead) {
    float total = 0;
    printf("\n===== BILL DETAILS =====\n");
    Order *curr = head;
    while (curr != NULL) {
        MenuItem *item = findMenuItemById(menuHead, curr->itemId);
        if (item && item->quantity >= curr->quantity) {
            float cost = item->price * curr->quantity;
            printf("  %d x %-15s = %.2f BDT\n",
                  curr->quantity, item->name, cost);
            total += cost;
            item->quantity -= curr->quantity; /* update stock */
        }
        curr = curr->next;
    }
    /* Apply 10% discount for orders above 500 BDT */
    if (total > 500) {
        printf("  Discount (10%%): -%.2f BDT\n", total * 0.10f);
        total *= 0.90f;
    }
    printf("  TOTAL : %.2f BDT\n", total);
    return total;
}

```

3.1.5 Customer Flow Controller

```

void customerFlow(MenuItem *menuHead,
                  Review **reviewList, Request **requestList) {
    welcomeCustomer();
    Order *orderList = NULL;
    int choice, qty;
    while (1) {
        displayMenu(menuHead);
        scanf("%d", &choice);
        if (choice == 0) { processOrders(...); collectReview(); break; }
        else if (choice == -1) { freeOrderList(&orderList); }
        else if (choice == -2) { requestNewItem(requestList); }
        else if (choice == -3) { showSpecialOffers(); }
        else {
            MenuItem *item = findMenuItemById(menuHead, choice);
            if (!item) { printf("Invalid item ID\n"); continue; }
            scanf("%d", &qty);
            appendOrder(&orderList, choice, qty);
        }
    }
}

```

3.1.6 Authority (Admin) Flow Controller

```

void authorityFlow(MenuItem **menuHead,
                  Review *reviewList, Request *requestList,
                  int *nextId) {
    if (!authorityLogin()) return; /* password gate */
    int option;
    while (1) {
        /* Print admin menu ... */
        scanf("%d", &option);
        if (option == 0) break;
        switch (option) {
            case 1: displayMenu(*menuHead); break;
            case 2: /* add item */ appendMenuItem(menuHead, ...); break;
            case 3: /* remove */ deleteMenuItem(menuHead, id); break;
            case 4: /* stock */ item->quantity = qty; break;
            case 5: displayReviews(reviewList); break;
            case 6: displayRequests(requestList); break;
        }
    }
}

```

3.1.7 Main Function

```

int main() {
    MenuItem *menuHead = NULL;
    int nextId = 6;
    appendMenuItem(&menuHead, 1, "Pizza", 250.0f, 10);
    appendMenuItem(&menuHead, 2, "Burger", 150.0f, 15);
    appendMenuItem(&menuHead, 3, "Pasta", 200.0f, 12);
    appendMenuItem(&menuHead, 4, "Sandwich", 120.0f, 20);
    appendMenuItem(&menuHead, 5, "Coffee", 80.0f, 30);

    Review *reviewList = NULL;
    Request *requestList = NULL;

    int role;
    scanf("%d", &role);
    if (role == 1) customerFlow(menuHead, &reviewList, &requestList);
    else if (role == 2) authorityFlow(&menuHead, reviewList, ..);

    /* --- Free all linked list memory --- */
    /* Free menuHead, reviewList, requestList nodes */
    return 0;
}

```

3.2 Performance Analysis

Operation	List	Time Complexity	Space Complexity
appendMenuItem	MenuItem	$O(n)$	$O(1)$
findMenuItemById	MenuItem	$O(n)$	$O(1)$
deleteMenuItem	MenuItem	$O(n)$	$O(1)$
appendOrder	Order	$O(n)$	$O(1)$
processOrders	Order $\times$ MenuItem	$O(n \cdot m)$	$O(1)$
freeOrderList	Order	$O(n)$	$O(n)$ freed
appendReview	Review	$O(n)$	$O(1)$
appendRequest	Request	$O(n)$	$O(1)$

All traversal operations are $O(n)$ where n is the number of nodes in the respective list. The processOrders function is $O(n \cdot m)$ where n is the number of ordered items and m is the menu size — this remains acceptable given typical restaurant menu and order sizes.

3.3 Results and Discussion

The system was tested across the following scenarios:

- Single-item order with no discount (total $\leq$ 500 BDT) — correct bill printed, stock decremented.
- Multi-item order exceeding 500 BDT — 10% discount correctly applied.
- Ordering an item with insufficient stock — system reports shortage without crashing.
- Admin adding a new item (ID auto-increments from 6) — item appears in subsequent menu display.
- Admin removing an item by ID — linked list correctly re-links prev and next nodes.
- Multiple sequential customer sessions — review and request lists accumulate correctly.
- Wrong admin password — access denied, returns to role selection.
- Ordering an invalid item ID — graceful error message, loop continues.

All test cases passed. Memory was verified free of leaks through manual code review of all malloc/free pairs.

Chapter 4: Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

Restaurant management systems reduce human error in order taking and billing. By automating stock tracking and order processing, staff can focus on service quality. Digital review collection provides immediate feedback loops that help management improve the dining experience.

4.1.2 Impact on Society & Environment

Digital ordering systems reduce paper waste (handwritten receipts, order pads). The system's real-time stock management reduces food over-ordering and waste — a meaningful environmental benefit given the scale of the restaurant industry. Accessible technology for small restaurants in Bangladesh supports inclusive economic growth.

4.1.3 Ethical Aspects

Customer data (reviews and food requests) is stored only in-memory for the session and is not persisted to disk, protecting user privacy. The admin authentication mechanism prevents unauthorized menu modifications, ensuring data integrity. The system does not collect personally identifiable information.

4.1.4 Sustainability Plan

- Extend to persistent file-based storage (CSV/binary) for multi-session data retention.
- Add multi-table support to scale from single-station to multi-POS environments.
- Implement a reporting module to analyze sales trends and optimize inventory purchasing.

4.2 Project Management and Team Work

The project followed a structured five-phase development plan:

1. Requirements gathering and system design (all members, Day 1–2).
2. Module-level implementation divided by data structure responsibility (Day 3–9).
3. Integration testing and edge-case debugging (all members, Day 10).
4. Documentation writing and report compilation (all members, Day 11–12).
5. Final review, code cleanup, and submission (all members, Day 13).

Regular team meetings ensured all modules integrated without naming conflicts. Version control was maintained manually through shared files.

4.3 Complex Engineering Problem

This project addresses a complex engineering problem (CEP) by requiring the design of a data-driven system with multiple interacting modules. The key engineering challenges solved include:

- Dynamic memory management — safe allocation and deallocation of linked list nodes without memory leaks.
- Pointer indirection — using double pointers (**head) to allow functions to modify list heads.
- Cross-list data integrity — order processing must validate against the menu list in real time.
- Role-based access — a single executable handles two entirely different workflows based on authenticated input.

4.3.1 Mapping of Program Outcome

PO	Description	Evidence in SRMS
PO3	Design solutions for complex engineering problems	Linked list CRUD for dynamic menu

PO5	Modern engineering tools usage	C standard library, dynamic memory
PO9	Individual and team project work	5-member team with defined roles

4.3.2 Complex Problem Solving

The `processOrders()` function exemplifies complex problem solving: it must traverse the Order list, cross-reference each order against the MenuItem list, validate stock availability, compute costs, apply conditional discount logic, update stock in-place, and display a formatted bill — all in a single linear pass.

4.3.3 Engineering Activities

Engineering activities demonstrated include: requirements specification, data structure selection (linked lists over arrays for dynamic sizing), algorithm design (append-traverse-delete), module integration, boundary-condition testing, and technical documentation.

Chapter 5: Conclusion

5.1 Summary

The Smart Restaurant Management System successfully demonstrates the practical application of singly linked lists in a real-world context. The system implements four independent linked lists — Menuitem, Order, Review, and Request — each supporting appropriate CRUD operations.

The dual-role architecture cleanly separates customer-facing and admin-facing functionality, with password-based authentication protecting sensitive operations. The billing system correctly handles multi-item orders, stock validation, and automatic discount application.

All learning objectives of CSE124: Data Structure Lab were met: dynamic memory allocation, pointer manipulation, linked list traversal, and modular C programming were all demonstrated in a cohesive, functional application.

5.2 Limitations

- No persistent storage — all data is lost when the program exits.
- Single-user console interface — no concurrent access support.
- Append-only Order traversal — no ability to remove a single item from an in-progress order.
- Linear search $O(n)$ for menu lookups — could be improved with hash tables for larger menus.
- Fixed-length char arrays — long names or feedback are silently truncated.

5.3 Future Work

- File I/O — serialize linked lists to binary files to achieve persistent storage across sessions.
- Hash table indexing — $O(1)$ average-case menu item lookup for large menus.
- Doubly linked list for Order — enable $O(1)$ item removal before checkout.
- Network layer — multi-table support with socket-based communication between waiter terminals.
- GUI frontend — ncurses-based terminal UI or Qt desktop interface.
- Encrypted password storage — replace plaintext AUTH_PASSWORD with hashed credential.

References

6. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
7. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
8. Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). Data Structures and Algorithms. Addison-Wesley.
9. Tanenbaum, A. S. (2007). Modern Operating Systems (3rd ed.). Prentice Hall.
10. ISO/IEC 9899:2018 — C18 Standard. International Organization for Standardization.
11. CSE124 Lab Manual — Data Structure Lab. Department of CSE, Daffodil International University, 2026.